

UrbanRisk Twin

Platformă cloud pentru analiza riscurilor urbane

Tema 5 – Raport tehnic

Cloud Computing

Membru echipă 1: Iurascu Vasile

Membru echipă 2: Tatarici Emanuel-Codrin

Facultate: Facultatea de Informatică, Iași

Data: May 19, 2026

Cuprins

1	Introducere	3
2	Studiu de caz și context real	3
2.1	Context internațional	3
2.2	Context național și local	3
3	Soluții existente și înrudite	3
3.1	Copernicus Land Monitoring	3
3.2	Date satelitare Sentinel-2	3
3.3	OpenAQ	4
3.4	Dashboard-uri smart city	4
3.5	API-uri de prognoză meteo	4
4	Soluția propusă	4
4.1	Descriere generală	4
4.2	Element de inovație	4
5	Tehnologii și motivare	5
5.1	React + Vite	5
5.2	Python + Flask	5
5.3	Firestore	5
5.4	Cloud Storage	5
5.5	Cloud Scheduler	5
5.6	OpenAPI / SwaggerHub	5
6	Arhitectura sistemului	6
6.1	Componente arhitecturale	8
6.2	Fluxul de date	8
7	Diagrama use-case și descrierea cazurilor de utilizare	8
7.1	Actori	9
7.2	Cazuri de utilizare principale	9
8	Fluxuri funcționale	11
8.1	Fluxul dashboard-ului	12
8.2	Fluxul de detalii ale zonei	12
8.3	Fluxul de simulare	12

9	Specificația API	12
9.1	Exemplu de răspuns API	13
9.2	Documentația OpenAPI pe SwaggerHub	13
10	Business Model Canvas	14
10.1	Propunerea de valoare în context cloud	14
11	Limitări	14
12	Concluzie	15
13	Bibliografie	15

1 Introducere

UrbanRisk Twin este o platformă cloud care evaluează principalele riscuri urbane din municipiul Iași, și anume caniculă, inundații și poluare, agregându-le într-un scor global per zonă. Proiectul reprezintă o demonstrație educațională de Cloud Computing și nu un sistem oficial de monitorizare.

2 Studiu de caz și context real

2.1 Context internațional

Orașele sunt tot mai afectate de valuri de căldură, insule de căldură urbană, precipitații intense și poluare. Există deja platforme specializate (Copernicus, OpenAQ, dashboard-uri smart city), însă fiecare acoperă doar o dimensiune. UrbanRisk Twin propune o integrare simplificată a mai multor indicatori urbani într-un singur dashboard.

2.2 Context național și local

Iași prezintă presiuni urbane tipice: trafic, spații verzi reduse, poluare locală și acumulări de apă la ploi abundente. Diversitatea zonelor (Copou, Bucium, Podu Roș, Centru, Nicolina, Tătărași) permite comparații relevante între profiluri de risc.

3 Soluții existente și înrudite

Proiectul propus se raportează la câteva categorii de sisteme deja existente, descrise pe scurt în subsecțiunile de mai jos.

3.1 Copernicus Land Monitoring

Serviciu european care oferă date deschise despre acoperirea terenurilor, vegetație și variabile de mediu, colectate atât din observații satelitare, cât și din surse. Pentru UrbanRisk Twin reprezintă o sursă posibilă pentru indicatorii de vegetație și suprafață, care în prezent sunt estimați manual în prototip.

3.2 Date satelitare Sentinel-2

Imagini satelitare multispectrale distribuite gratuit prin Copernicus, cu rezoluție potrivită pentru analiza fenomenelor urbane. Pot susține în viitor estimări mai exacte ale vegetației și densității urbane per zonă, înlocuind valorile sintetice utilizate la calculul riscurilor.

3.3 OpenAQ

Platformă open data care agregă măsurători de calitate a aerului provenite de la stații oficiale și senzori comunitari, expunându-le printr-un API public. Reprezintă o sursă naturală pentru indicatorii de poluare folosiți în calculul scorului, oferind acoperire uniformă indiferent de furnizor.

3.4 Dashboard-uri smart city

Platforme municipale care afișează date urbane brute pe categorii precum trafic, calitatea aerului sau consum energetic, fără a le combina într-un indicator unic. UrbanRisk Twin păstrează aceeași logică de vizualizare interactivă, dar se diferențiază prin agregarea explicită a riscurilor într-un scor compus și prin posibilitatea de a rula simulări “what-if”.

3.5 API-uri de prognoză meteo

Servicii naționale sau comerciale care furnizează date de temperatură, umiditate, vânt și precipitații, cu acoperire geografică fină și actualizări frecvente. Pot alimenta automat recalcularea scorurilor de caniculă și inundații prin Cloud Scheduler, atunci când apar valori extreme sau modificări semnificative ale prognozei.

4 Soluția propusă

4.1 Descriere generală

UrbanRisk Twin este o platformă web compusă dintr-un frontend React, un API REST în Flask, un motor de calcul al riscurilor și servicii de date pregătite pentru cloud. Sistemul produce patru scoruri per zonă: risc de caniculă, risc de inundații, risc de poluare și risc global, pe o scară de la 0 la 100, asociate cu etichetele Scăzut / Mediu / Ridicat.

Dashboard-ul permite utilizatorilor să inspecteze toate zonele, să detalieze o anumită zonă, să ruleze simulări de scenarii și să citească recomandări bazate pe scoruri.

4.2 Element de inovație

Elementul inovator constă în combinarea mai multor dimensiuni ale riscului urban într-o singură platformă cloud explicabilă. În loc să afișeze doar date brute, sistemul prezintă scoruri și recomandări. Utilizatorii pot rula și simulări de scenarii (de exemplu, un val de căldură sau o ploie abundentă) și pot observa cum se modifică profilurile de risc, transformând dashboard-ul într-un instrument ușor de suport decizional.

5 Tehnologii și motivare

Tehnologiile alese pentru UrbanRisk Twin sunt descrise pe scurt în subsecțiunile de mai jos.

5.1 React + Vite

React este un framework de frontend matur, potrivit în cazul de față pentru dashboard-uri interactive și componente UI reutilizabile, cu un ecosistem bogat de biblioteci complementare. Vite este folosit ca build tool, oferind un server de dezvoltare rapid și o configurare minimală pentru proiectele React.

5.2 Python + Flask

Python a fost ales pentru logica de calcul și procesarea datelor din motorul de risc, datorită expresivității și a bibliotecilor disponibile pentru manipularea datelor. Flask completează stack-ul ca framework backend și este potrivit pentru expunerea unui API REST.

5.3 Firestore

Document store gestionat din Google Cloud, potrivit pentru date semi-structurate despre zone și profiluri de risc. Permite scalare automată și acces din mai multe servicii, eliminând necesitatea administrării unui server de bază de date.

5.4 Cloud Storage

Serviciu de stocare de obiecte folosit pentru date brute, exporturi JSON/CSV și rapoarte generate. Oferă durabilitate ridicată și acces uniform prin API, ceea ce simplifică schimbul de fișiere între jobs și frontend.

5.5 Cloud Scheduler

Serviciu de cron gestionat care declanșează periodic recalcularea scorurilor de risc, fără a necesita un server dedicat. Permite definirea unor reguli flexibile pentru actualizările automate ale dashboard-ului.

5.6 OpenAPI / SwaggerHub

OpenAPI este un mod standard de a descrie API-uri REST, iar SwaggerHub oferă o platformă pentru publicarea și vizualizarea interactivă a specificației. Această combinație

facilitează atât consumarea API-ului de către frontend, cât și posibila generare automată de clienți și schelete de cod.

6 Arhitectura sistemului

Arhitectura este modulară și pregătită pentru cloud. Frontend-ul comunică cu backend-ul prin HTTP; backend-ul expune endpoint-uri REST și delegă calculele către motorul de risc. Datele sunt citite din fișiere locale în timpul dezvoltării și din Firestore în versiunea cloud.

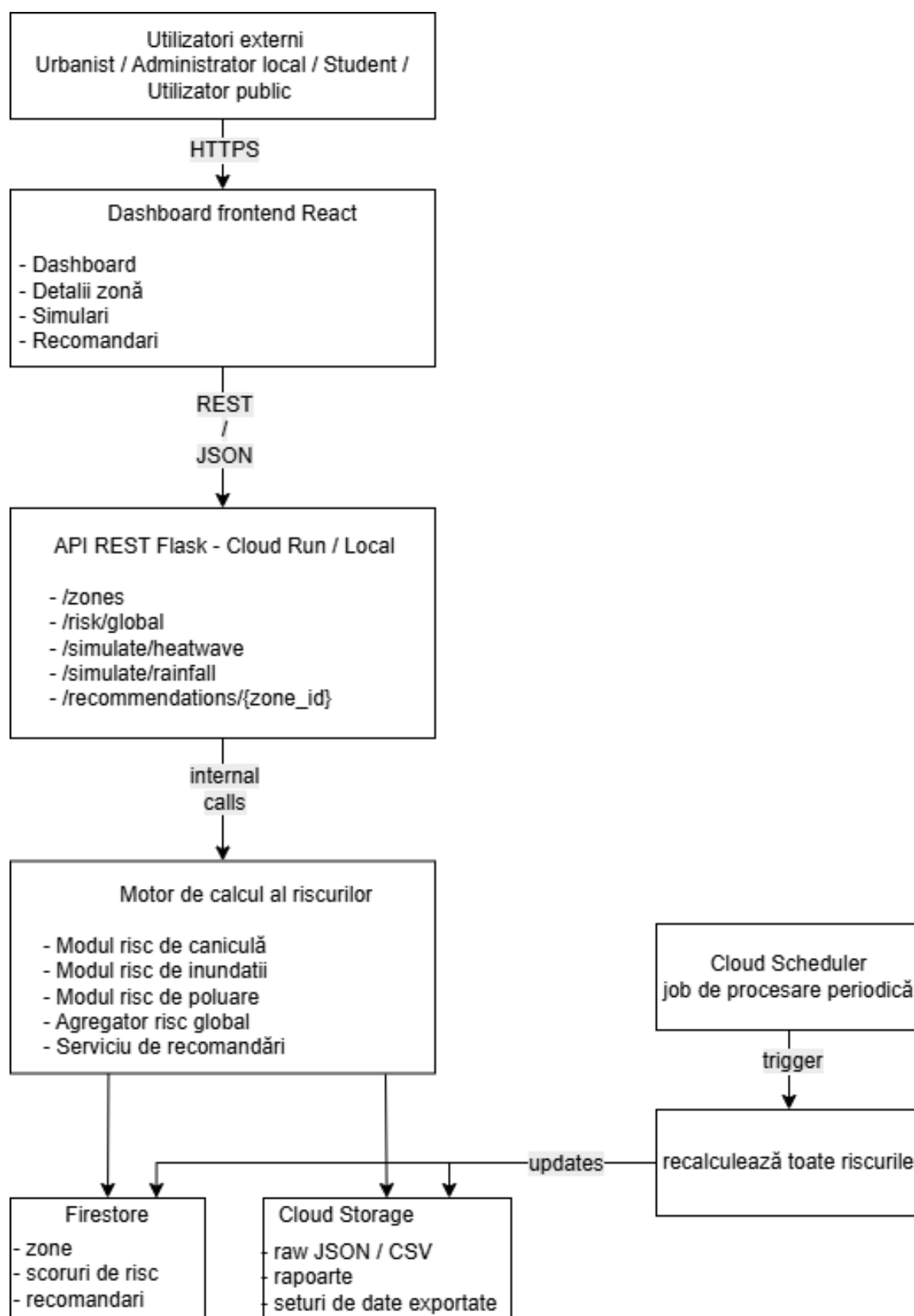


Figura 1: Arhitectura propusă a sistemului UrbanRisk Twin.

6.1 Componente arhitecturale

- **Dashboard React** – interfața pentru zone, scoruri, simulări și recomandări.
- **API REST Flask** – endpoint-urile consumate de frontend.
- **Motor de risc** – logica de calcul pentru risc de caniculă, inundații, poluare și risc global.
- **Strat de servicii de date** – abstractizează accesul la fișiere JSON locale sau la Firestore.
- **Firestore** – stochează zonele și profilurile de risc în versiunea cloud.
- **Cloud Storage** – stochează seturi de date și rapoarte generate.
- **Cloud Scheduler** – declanșează recalcularea periodică.
- **Cloud Run** – găzduiește backend-ul containerizat.

6.2 Fluxul de date

Fluxul la cerere: utilizatorul deschide dashboard-ul, React apelează API-ul Flask, backend-ul citește datele zonelor și calculează scorurile prin motorul de risc, iar API-ul returnează JSON pe care frontend-ul îl afișează.

Fluxul programat: Cloud Scheduler declanșează un job de procesare, backend-ul încarcă datele cele mai recente, motorul de risc recalculează toate scorurile, iar rezultatele sunt salvate înapoi în Firestore pentru a fi afișate în dashboard.

7 Diagrama use-case și descrierea cazurilor de utilizare

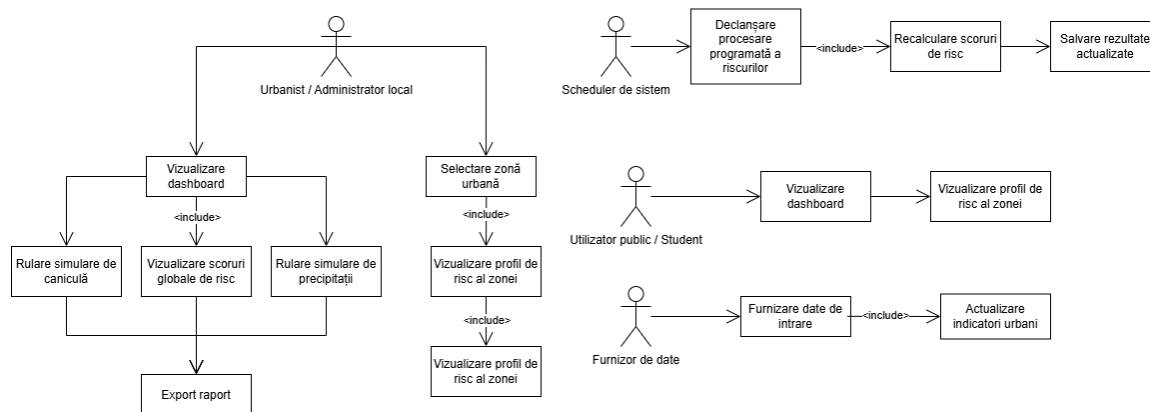


Figura 2: Diagrama use-case pentru UrbanRisk Twin.

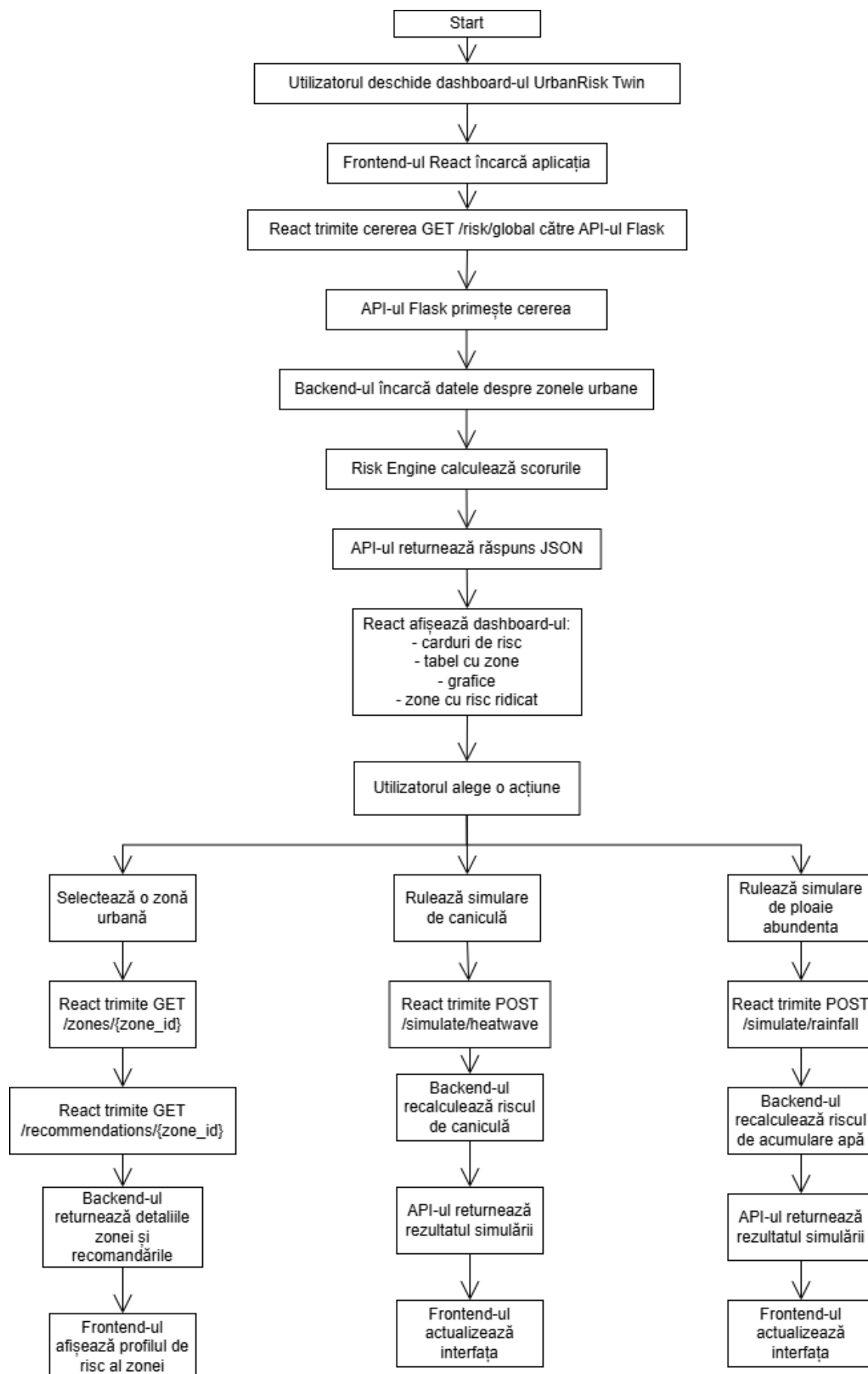
7.1 Actori

- **Urbanist / administrator local** – analizează zonele de risc și recomandările.
- **Utilizator public** – consultă informații generale despre risc.
- **Scheduler de sistem** – componentă automatizată pentru procesare periodică.
- **Furnizor de date** – sursă externă de date meteorologice sau urbane.

7.2 Cazuri de utilizare principale

Caz de utilizare	Descriere
Vizualizare dashboard	Privire de ansamblu asupra tuturor zonelor, riscului mediu, zonelor cu risc ridicat și a graficelor.
Selectarea unei zone urbane	Inspectarea indicatorilor și a scorurilor de risc pentru o zonă specifică.
Vizualizarea profilului de risc	Afișarea riscurilor de caniculă, inundații, poluare și a riscului global pentru zona selectată.
Rulare simulare caniculă	Utilizatorul oferă temperatura și umiditatea; riscul de caniculă este recalculat.
Rulare simulare precipitații	Utilizatorul oferă o valoare a precipitațiilor; riscul de inundații este recalculat.
Vizualizarea recomandărilor	Recomandări bazate pe scoruri pentru zona selectată.
Declanșarea procesării programate	Scheduler-ul rulează jobul backend pentru actualizarea scorurilor de risc.
Export raport	Funcționalitate viitoare: export PDF sau CSV al rezultatelor selectate.

8 Fluxuri funcționale



8.1 Fluxul dashboard-ului

Utilizatorul deschide aplicația React, care apelează `GET /risk/global`. Flask încarcă zonele, motorul de risc produce valorile, API-ul returnează JSON, iar React afișează carduri, tabele și grafice.

8.2 Fluxul de detalii ale zonei

Utilizatorul selectează o zonă. React apelează `GET /zones/{zone_id}` și `GET /recommendations/{zone_id}`. Backend-ul returnează ambele, iar frontend-ul afișează profilul complet de risc împreună cu recomandările.

8.3 Fluxul de simulare

Utilizatorul introduce o valoare simulată pentru temperatură sau precipitații. React trimite o cerere POST către endpoint-ul de simulare, backend-ul recalculează scorul de risc afectat, iar frontend-ul afișează rezultatele actualizate.

9 Specificația API

Backend-ul expune un API REST proiectat conform specificațiilor OpenAPI și documentat pe SwaggerHub. Principalele endpoint-uri sunt rezumate mai jos.

Metodă	Endpoint	Descriere
GET	/	Stare și metadate ale aplicației.
GET	/zones	Toate zonele urbane analizate.
GET	/zones/{zone_id}	Detalii pentru o anumită zonă.
GET	/risk/heat	Scorurile de risc de caniculă pentru toate zonele.
GET	/risk/flood	Scorurile de risc de inundații pentru toate zonele.
GET	/risk/pollution	Scorurile de risc de poluare pentru toate zonele.
GET	/risk/global	Profilurile complete de risc și scorurile globale.
GET	/recommendations/{zone_id}	Recomandări pentru o zonă.
POST	/simulate/heatwave	Simulare caniculă (temperatură, umiditate).
POST	/simulate/rainfall	Simulare precipitații (cantitate de ploaie).

Metodă	Endpoint	Descriere
POST	/process-risks	Declanșează recalcularea tuturor scorurilor.

9.1 Exemplu de răspuns API

```
{
  "zone_id": "tatarasi_01",
  "name": "Tatarasi",
  "city": "Iasi",
  "heat_risk": 74,
  "flood_risk": 55,
  "pollution_risk": 72,
  "global_risk": 68,
  "heat_label": "High",
  "flood_label": "Medium",
  "pollution_label": "High",
  "global_label": "High"
}
```

9.2 Documentația OpenAPI pe SwaggerHub

Specificația OpenAPI completă pentru API-ul UrbanRisk Twin este publicată pe SwaggerHub:

<https://app.swaggerhub.com/apis/nocompany-df5/UrbanRisk/1.0.0>

Specificația acoperă toate endpoint-urile prezentate în tabelul de mai sus, inclusiv parametrii, corpurile cererilor și schemele de răspuns.

10 Business Model Canvas

PARTENERI CHEIE ● Primăria Iași ● ANM (date meteo) ● OpenStreetMap ● Universități ● Copernicus / EEA ● Google Cloud (credite edu)	ACTIVITĂȚI CHEIE ● Calcul scoruri risc ● Ingestie date meteo / trafic ● Mentenanță cloud ● Evangelizare academică	PROPUNERE DE VALOARE ● Vizualizare unificată a riscurilor urbane (caniculă, inundații, poluare) ● Scor compus per zonă ● Simulare scenarii “what-if” ● Suport decizional educațional ● Open data + open source ● Focus România / Iași	RELĂȚII CU CLIEȚII ● Self-service prin dashboard ● Comunitate GitHub	SEGMENTE DE CLIEȚI ● Autorități locale (primării) ● Cercetători urbani ● Studenți, ONG-uri de mediu ● Jurnaliști de specialitate ● Cetățeni informați
	RESURSE CHEIE ● Credite Google Cloud ● Date open (OSM, Copernicus) ● Cod open source ● Echipă tehnică		CANALE ● Web app public ● API public (SwaggerHub) ● Conferințe academice ● Articole de blog	
STRUCTURA COSTURILOR ● Infrastructură cloud: Cloud Run, Firestore, Cloud Storage, Scheduler ● Monitorizare și logging			SURSE DE VENIT ● Grant-uri academice / fonduri europene de adaptare climatică ● Sponsorizări de la primării ● Versiune “Pro”: rapoarte custom pentru autorități ● Workshop-uri plătite și training pentru decidenți	

Figura 4: Business Model Canvas pentru UrbanRisk Twin (notație Strategyzer).

10.1 Propunerea de valoare în context cloud

Contextul cloud permite platformei să scaleze de la un prototip local la o soluție multi-orăș, oferind procesare periodică a datelor, API-uri găzduite și stocare gestionată, astfel încât utilizatorii nu trebuie să administreze infrastructură fizică.

11 Limitări

- Setul de date inițial poate conține valori simulate.
- Formulele de risc sunt simplificate, în scop educațional.
- Platforma nu oferă alerte oficiale de mediu sau de urgență.
- Indicele de vegetație și densitatea urbană sunt estimate manual în prototip.
- Recomandările sunt bazate pe reguli, nu pe învățare automată.

Aceste limitări sunt acceptabile pentru o primă versiune focalizată pe demonstrarea arhitecturii cloud, a designului modular și a modelării explicabile a riscului.

12 Concluzie

UrbanRisk Twin propune o abordare cloud pentru analiza riscurilor urbane: mai mulți indicatori sunt combinați în scoruri explicabile și expuși printr-un dashboard interactiv. Proiectul este relevant pentru Cloud Computing deoarece utilizează o arhitectură distribuită (frontend, backend, stocare, procesare programată și documentație API).

13 Bibliografie

Bibliografie

- [1] Copernicus Data Space Ecosystem.
<https://dataspace.copernicus.eu/>
- [2] OpenAQ Platform.
<https://openaq.org/>
- [3] Documentația Google Cloud Run.
<https://cloud.google.com/run/docs>
- [4] Documentația Cloud Firestore.
<https://firebase.google.com/docs/firestore>
- [5] OpenAPI Specification.
<https://spec.openapis.org/oas/latest.html>
- [6] Documentația React.
<https://react.dev/>
- [7] Documentația Flask.
<https://flask.palletsprojects.com/>